STEP 3 · 3주차

평가와 임계값 설계

두 개의 자로 재고, 클래스마다 다른 기준을 정한다

강의 50분 | 용어 정리 | 실습 6단계

이 논문의 핵심 아이디어는 여기 있다. 경계 정확도가 낮아도 쓸모 있는 시스템을 만드는 법.

이번 주 진행 방식

마지막 주

① 강의	50분	mask mAP · 이미지 단위 ON/OFF 평가 · 클래스별 임계값	본 강의 자료 28~41페이지
② 용어 정리	20분	평가 지표와 임계값 용어	이 자료 4~7페이지
③ 실습	70분	성능 평가 → 검증셋 임계값 탐색 → 테스트셋 고정 적용 → 오류 분석	이 자료 9~15페이지
④ 최종 제출	3주차 자정	최종 리포트 · 실습 코드 · 결과 파일	이 자료 18페이지

지난주 전체 학습을 완료했다면 best.pt를 불러와 평가부터 시작한다.

오늘 강의에서 기억할 여섯 가지

실습 전에 다시 확인

1	**두 개의 평가 축**	마스크 경계 정확도(mAP)와 영상 선별 능력(ON/OFF)은 다른 것을 잰다
2	**mAP가 낮은 이유**	균열은 폭이 몇 픽셀이라 1~2픽셀만 어긋나도 IoU가 급락한다
3	**ON/OFF 평가**	영상에 그 결함이 있는지 없는지만 판정한다. 실무 선별에 맞는 기준
4	**클래스별 임계값**	세 결함의 신뢰도 분포가 달라 하나의 공통 임계값으로는 부족하다
5	**절차가 핵심**	임계값은 검증셋에서 정하고 테스트에는 그대로 적용한다
6	**논문 결과**	mask mAP50 0.443 · 테스트 macro F1 0.896

다섯 번째 줄이 이번 주 실습의 규칙이다. 테스트 결과를 보고 임계값을 고치면 실험이 무효가 된다.

용어 정리 ① 겹침 지표

예측 마스크와 정답 마스크가 얼마나 잘 겹치는가

IoU Intersection over Union	예측 영역과 정답 영역의 교집합을 합집합으로 나눈 값 → 0~1 · 1이면 완벽히 일치
AP Average Precision	정밀도-재현율 곡선 아래 면적 → 클래스 하나에 대한 값
mAP50 –	IoU 0.50 기준 클래스별 AP의 평균 → IoU 0.50 기준에서 결함 영역을 얼마나 잘 검출·분할했는가
mAP50-95 –	IoU 0.50~0.95를 0.05 간격으로 평균 → 다양한 IoU 기준에서 위치와 영역 일치도를 종합적으로 평가
mask 기준 –	박스가 아니라 마스크로 계산 → 손상 영역 자체를 평가

가는 균열처럼 폭이 좁은 객체는 예측 마스크가 몇 픽셀만 어긋나도 IoU가 크게 감소할 수 있다.

용어 정리 ② ON/OFF 평가

이 논문의 핵심 아이디어

ON/OFF 평가 Image-level screening	영상에 해당 클래스의 결함이 존재하는지 여부를 판정 → 결함 포함 영상을 우선 선별하는 능력을 평가
정답 ON Ground truth ON	해당 클래스의 Ground Truth 인스턴스가 하나 이상 존재하면 ON → 영상 단위 판정
예측 ON Prediction ON	해당 클래스의 예측 중 confidence가 설정 임계값 이상인 인스턴스가 하나라도 있으면 ON → 임계값이 여기서 작동
TP·FP·FN·TN –	영상 단위의 TP·FP·FN·TN을 클래스별로 계산 → 클래스별로 센다
macro F1 –	세 클래스 F1의 평균 → 전체 성능 요약 지표

마스크 경계가 완전히 일치하지 않더라도, 해당 결함이 포함된 영상을 올바르게 선별하면 영상 단위 평가에서는 정답으로 판정한다.

용어 정리 ③ 임계값

낮추면 얻는 것과 잃는 것

신뢰도 Confidence	모델이 해당 예측에 부여한 confidence score → 0~1 사이 값
임계값 Threshold	예측을 채택할 최소 신뢰도 → 이보다 낮으면 버린다
임계값을 낮추면 –	더 많은 예측을 허용 → Recall이 증가할 수 있지만 FP도 증가할 수 있음
임계값을 높이면 –	높은 confidence 예측만 유지 → FP는 줄 수 있지만 FN이 증가할 수 있음
정보 누출 Information leakage	테스트셋 결과를 보고 임계값을 다시 조정하는 것 → 테스트셋 독립성이 훼손되고 성능이 과대평가될 수 있음

crack = 0.010, efflorescence = 0.004, spall_detach = 0.060 — 검증셋에서 클래스별 F1 정점이 서로 다르게 나타나 클래스별 임계값을 적용하였다.

용어 정리 ④ 오류 분석

왜 틀렸는지 설명하기

오탐 False Positive	실제 결함이 없는 영역을 결함으로 예측 → 시공 이음부, 케이블, 그림자 등 비결함 요소가 원인이 될 수 있음
미탐 False Negative	실제 결함이 존재하지만 모델이 검출하지 못함 → 가는 균열, 저대비 백태, 초기 spall_detach 등에서 발생 가능
질감 대비 Texture contrast	결함과 주변 배경의 시각적 차이 정도 → 대비가 낮을수록 검출이 어려워질 수 있음
확산적 경계 Diffuse boundary	결함의 경계가 명확하지 않고 점진적으로 퍼지는 형태 → efflorescence에서 자주 나타남
주석 모호성 Annotation ambiguity	경계가 불명확한 결함은 주석자에 따라 라벨 범위가 달라질 수 있음 → IoU 및 mAP 평가에 영향

예측 영역이 시각적으로 타당해 보여도 Ground Truth와 경계가 다를 수 있다. 이러한 사례는 주석 기준을 재검토하고 정교화하는 근거가 된다.

오늘의 실습

여섯 단계 — 지난주 학습 결과에서 시작한다

단계 1	**세그멘테이션 성능 평가**	mask mAP50 및 mAP50-95 산출
단계 2	**ON/OFF 평가 구현**	클래스별 영상 단위 판정 함수 작성
단계 3	**검증셋에서 클래스별 임계값 탐색**	임계값별 F1 곡선 작성
단계 4	**테스트셋 고정 적용**	검증셋에서 선택한 임계값을 테스트셋에 그대로 적용
단계 5	**오류 사례 분석**	FP·FN 사례를 확인하고 오탐·미탐 원인을 분석
단계 6	**리포트 작성**	평가 결과·임계값·오류 분석을 정해진 구조로 정리

단계 3의 검증셋 기반 임계값 탐색이 이번 주의 핵심이다. 임계값은 검증셋에서 결정하고 테스트셋에는 변경 없이 적용한다.

세그멘테이션 성능 측정

지난주 best.pt 를 불러온다

목표 **테스트셋에서 mask mAP50과 mAP50-95를 구한다**

```
from pathlib import Path
from ultralytics import YOLO

# 현재 실습 폴더 기준 경로
BASE_DIR = Path.cwd()

DATA_YAML = BASE_DIR / "data.yaml"
BEST_PT = BASE_DIR / "runs" / "step3" / "weights" / "best.pt"

# 파일 존재 확인
if not DATA_YAML.exists():
    raise FileNotFoundError(f"data.yaml을 찾을 수 없습니다: {DATA_YAML}")

if not BEST_PT.exists():
    raise FileNotFoundError(f"best.pt를 찾을 수 없습니다: {BEST_PT}")
```

확인 방법

- 절대적인 정상 범위는 없으므로, 이전 학습 결과 및 클래스별 성능과 비교해 해석한다
- 본 연구의 테스트 mask mAP50은 0.443
- 다른 데이터셋·과제의 mAP와 단순 비교하지 말고, 동일 조건과 연구 목적을 기준으로 해석한다
- 클래스별 성능 차이를 데이터 수, 결함 크기, 경계 명확성 등의 관점에서 분석한다

세그멘테이션 성능 측정 코드(2)

```
# 모델 불러오기
model = YOLO(str(BEST_PT))

# 테스트셋 평가
results = model.val(
    data=str(DATA_YAML),
    split="test",
    imgsz=960,
    plots=True
)

# 전체 성능
print("mask mAP50   :", round(results.seg.map50, 4))
print("mask mAP50-95:", round(results.seg.map, 4))
print("box mAP50    :", round(results.box.map50, 4))
```

```
# 클래스별 mask mAP50-95
class_names = [
    "crack",
    "efflorescence",
    "spall_detach"
]

for i, name in enumerate(class_names):
    print(
        name,
        "mask mAP50-95:",
        round(results.seg.maps[i], 4)
    )
```

ON/OFF 평가 구현

논문의 핵심 아이디어를 코드로

목표 **클래스별 임계값을 받아 TP·FP·FN·TN을 세는 함수를 만든다**

```
import numpy as np

def onoff(model, paths, gts, thr, cid_target):
    tp = fp = fn = tn = 0

    for p, gt in zip(paths, gts):

        # 예측 수행
        r = model.predict(
            p,
            imgsz=960,
            verbose=False
        )[0]

        # 예측 클래스와 confidence
        conf = r.boxes.conf.cpu().numpy()
        cid = r.boxes.cls.cpu().numpy().astype(int)
```

확인 방법

- paths에는 평가할 이미지 경로 목록을 준비
- gts에는 각 이미지에 존재하는 정답 클래스 ID 목록을 준비
- 한 영상에 여러 클래스가 동시에 존재할 수 있음
- cid_target은 평가할 클래스 ID를 의미 0=crack, 1=effloresence, 2=spall_detach
- 먼저 thr=0.5로 실행해 함수가 정상 동작하는지 확인
- 이후 임계값 탐색은 **검증셋에서만** 수행

ON/OFF 평가 구현 코드(2)

논문의 핵심 아이디어를 코드로

```
# 해당 클래스의 예측 중 threshold 이상이 하나라도 있으면 ON
pred_on = bool(
    ((cid == cid_target) & (conf >= thr)).any()
)

# 정답 라벨에 해당 클래스가 있으면 ON
gt_on = cid_target in gt

# TP / FP / FN / TN 계산
if gt_on and pred_on:
    tp += 1
elif not gt_on and pred_on:
    fp += 1
elif gt_on and not pred_on:
    fn += 1
else:
    tn += 1
```

```
precision = tp / (tp + fp) if (tp + fp) else 0
    recall = tp / (tp + fn) if (tp + fn) else 0
    f1 = (
        2 * precision * recall / (precision + recall)
        if (precision + recall)
        else 0
    )

    return {
        "tp": tp,
        "fp": fp,
        "fn": fn,
        "tn": tn,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

임계값 탐색 — 검증셋에서만

이번 주에서 가장 중요한 단계

목표 **검증셋에서 클래스별 최적 임계값을 찾는다**

```
import numpy as np
import matplotlib.pyplot as plt

THRS = [0.001, 0.004, 0.01, 0.02, 0.06, 0.1, 0.2, 0.3]

class_names = [
    "crack",
    "efflorescence",
    "spall_detach"
]

best = {}
all_scores = {}

for cid, name in enumerate(class_names):
    scores = []
```

확인 방법

- 반드시 검증셋(val)만 사용한다
- 최고 F1이 동점이면 낮은 임계값을 선택해 Recall을 우선할 수 있다
- 본 연구의 선택 임계값: crack 0.010 / efflorescence 0.004 / spall_detach 0.060
- 클래스별 F1 곡선을 저장하고 선택 임계값의 근거로 사용

임계값 탐색 — 검증셋에서만 코드(2)

```
for t in THRS:
        r = onoff(
            model,
            val_imgs,
            val_gts,
            t,
            cid
        )

        scores.append(r["f1"])

        print(
            f"{name} | thr={t:.3f} | "
            f"F1={r['f1']:.3f} | "
            f"FP={r['fp']} | FN={r['fn']}"
        )

    all_scores[name] = scores

    # 최고 F1
    max_f1 = max(scores)

     # 동점이면 더 낮은 threshold 선택
    best_idx = next(
        i for i, s in enumerate(scores)
        if s == max_f1
    )
```

```
    best[name] = THRS[best_idx]

    print(
        f">> {name} 선택 임계값: "
        f"{THRS[best_idx]} "
        f"(F1={max_f1:.3f})\n"
    )

print("최종 임계값:", best)

# F1 곡선 시각화
for name in class_names:
    plt.plot(
        THRS,
        all_scores[name],
        marker="o",
        label=name
    )

plt.xlabel("Confidence threshold")
plt.ylabel("F1-score")
plt.title("Validation F1 by Threshold")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("val_threshold_f1.png", dpi=150)
plt.show()
```

테스트셋 고정 적용

절차를 지키는 것이 성능보다 중요하다

목표 **검증셋에서 정한 임계값을 테스트셋에 그대로 적용해 최종 성능을 낸다**

```
import pandas as pd

class_names = [
    "crack",
    "efflorescence",
    "spall_detach"
]

rows = []

for cid, name in enumerate(class_names):
    r = onoff(
        model,
        test_imgs,
        test_gts,
        best[name],   # 검증셋에서 선택한 임계값 그대로 사용
        cid
    )
```

확인 방법

- 검증셋에서 선택한 클래스별 임계값을 그대로 사용
- 테스트셋 결과를 보고 임계값을 다시 조정하지 않음
- 클래스별 TP·FP·FN·TN과 F1을 확인
- 세 클래스 F1의 평균으로 macro F1 계산
- 본 연구의 테스트 macro F1 참고값: **0.896**

테스트셋 고정 적용 코드 (2)

```
rows.append([
        name,
        best[name],
        r["tp"],
        r["fp"],
        r["fn"],
        r["tn"],
        r["precision"],
        r["recall"],
        r["f1"]
    ])
```

```
df = pd.DataFrame(
    rows,
    columns=[
        "class",
        "threshold",
        "TP",
        "FP",
        "FN",
        "TN",
        "precision",
        "recall",
        "f1"
    ]
)

print(df.round(3))
print("macro F1:", round(df["f1"].mean(), 3))
```

오류 사례 분석

숫자보다 그림이 설득력 있다

목표 클래스별 FP·FN 영상을 추출하고 오류 원인을 분석한다

```
from pathlib import Path
import cv2

class_names = [
    "crack",
    "efflorescence",
    "spall_detach"
]

SAVE_DIR = Path("error_cases")
SAVE_DIR.mkdir(exist_ok=True)

for cid, name in enumerate(class_names):

    fp_imgs = []
    fn_imgs = []
```

확인 방법

- 클래스별 FP·FN 영상을 추출하고 오류 원인을 분석한다
- 시공 이음부, 케이블, 그림자, 오염 등 비결함 요소와의 혼동 여부를 확인
- 결함 크기, 대비, 경계 명확성 등 미탐 원인을 확인
- 클래스별 대표 FP·FN 사례를 선정해 원인을 서술

오류 사례 분석 코드(2)

```
for p, gt in zip(test_imgs, test_gts):

        r = model.predict(
            p,
            imgsz=960,
            verbose=False
        )[0]

        conf = r.boxes.conf.cpu().numpy()
        pred_cls = r.boxes.cls.cpu().numpy().astype(int)

        # 검증셋에서 정한 클래스별 임계값 사용
        pred_on = bool(
            ((pred_cls == cid) & (conf >= best[name])).any()
        )

        gt_on = cid in gt

        if pred_on and not gt_on:
            fp_imgs.append(p)

        if not pred_on and gt_on:
            fn_imgs.append(p)
```

```
print(
        f"{name} | FP={len(fp_imgs)} | FN={len(fn_imgs)}"
    )

    # 클래스별 저장 폴더
    class_dir = SAVE_DIR / name
    class_dir.mkdir(exist_ok=True)

    # 대표 FP/FN 최대 3장씩 저장
    for error_type, paths in [
        ("FP", fp_imgs),
        ("FN", fn_imgs)
    ]:
        for i, p in enumerate(paths[:3]):

            r = model.predict(
                p,
                imgsz=960,
                verbose=False
            )[0]

            out_path = (
                class_dir /
                f"{error_type}_{i+1}_{Path(p).name}"
            )

            cv2.imwrite(
                str(out_path),
                r.plot()
            )
```

리포트 작성

마지막 단계

목표 **세그멘테이션 성능과 영상 단위 ON/OFF 평가 결과, 오류 분석을 하나의 리포트로 정리한다**

1. 문제 정의	세 결함의 정의 · 세그멘테이션을 선택한 이유 · 평가 목표
2. 데이터	주석 기준 요약 · 데이터 규모 · 클래스 구성 · 원본 단위 분할 방식
3. 방법	YOLOv8s-Seg 학습 설정 · imgsz 960 · Mosaic/MixUp 비활성화 근거
4. 결과	mask mAP50·mAP50-95 + ON/OFF 결과표 + 검증셋 임계값 F1 곡선
5. 오류 분석	클래스별 FP·FN 사례와 발생 원인 · 결함별 오류 특성
6. 한계	테스트 규모 · 물리 단위 정량화 불가 · 단일 모델
7. 재현 정보	실행 환경 · seed · data.yaml · 모델 설정 · 코드 실행 순서

여기까지 됐으면 정상

스스로 확인하는 체크포인트

오늘 안에 확인할 것

- mask mAP50과 mAP50-95를 산출했다
- 클래스별 ON/OFF 평가 함수를 구현하고 정상 동작을 확인했다
- 검증셋에서만 임계값을 탐색했다
- 검증셋 F1 곡선을 통해 클래스별 최적 임계값이 서로 다름을 확인했다
- 검증셋에서 선택한 임계값을 변경하지 않고 테스트셋에 적용했다
- 클래스별 FP·FN 사례를 추출하고 오탐·미탐 원인을 분석했다
- 테스트셋 규모와 그에 따른 성능 해석의 한계를 명시했다

③ 검증셋에서만 임계값을 탐색하고, ⑤ 선택한 임계값을 테스트셋에 그대로 적용하는 절차가 이번 과제의 핵심이다. 코드 실행 순서로 이를 확인할 수 있어야 한다.

자주 나는 오류와 해결

평가 단계의 함정들

증상	원인	해결
mAP가 0에 가까움	라벨 포맷, 클래스 ID, 좌표 범위 또는 데이터 경로 오류	2주차 무결성 점검과 라벨 예시를 다시 확인
ON/OFF F1이 0	gt_classes 생성 오류 또는 정답 클래스 매핑 불일치	라벨 파일에서 영상별 정답 클래스 목록을 다시 생성하고 클래스 ID를 확인
임계값을 바꿔도 F1 동일	예측 confidence 분포가 탐색 구간 밖에 몰려 있을 가능성	confidence 분포를 확인하고 필요한 경우 탐색 임계값 범위를 조정
테스트 성능이 검증보다 훨씬 높음	검증·테스트 표본 수가 적거나 데이터 분포 차이, 분할 오류 가능성	원본 단위 분할과 클래스 분포를 다시 확인
예측이 하나도 안 나옴	임계값 과다, 모델 로드 오류, 입력/라벨 설정 문제 가능성	낮은 임계값부터 확인하고 best.pt, 입력 경로, 클래스 설정 점검
추론이 매우 느림	GPU 미사용 또는 고해상도 입력으로 연산량 증가	GPU 사용 여부 확인, 필요 시 실습용으로 imgsz 축소

임계값 변화에도 F1이 거의 변하지 않는다면 먼저 예측 confidence 분포를 확인한다. 분포를 확인한 뒤 탐색 범위를 조정해야 한다.

최종 제출물

Step 3 마무리

① 리포트	문제 정의·데이터·방법·결과·오류 분석·한계·재현 정보를 포함한 PDF 1개
② 주석 기준서	1주차에 만든 문서 (수정본)
③ 세그멘테이션 성능	테스트셋 mask mAP50 · mAP50-95 및 클래스별 성능
④ 임계값 F1 곡선	검증셋 클래스별 F1 곡선 + 선택한 임계값과 선택 근거
⑤ ON/OFF 결과표	테스트셋 클래스별 TP·FP·FN·TN·Precision·Recall·F1 + macro F1
⑥ 오류 사례	클래스별 대표 FP·FN 사례 + 오류 원인 분석
제출 기한	3주차 강의일 자정

④ 임계값은 검증셋에서만 탐색하고, 테스트셋에는 변경 없이 적용했음을 코드와 결과로 확인할 수 있어야 한다.

Step 3을 마치며

무엇을 얻었는가

3주 동안 익힌 것

- 주석 기준서를 작성하고 일관되게 라벨링
- 원본 단위 분할로 데이터 누수 방지
- YOLOv8s-Seg 학습 및 검증 성능 기반 모델 선택
- mask mAP와 이미지 단위 ON/OFF 평가를 구분해 사용
- 검증셋 기반 클래스별 임계값 최적화 절차 이해

Step 4로 이어지는 것

- 데이터 누수 방지 원칙
- 검증셋에서 결정하고 테스트셋에 고정 적용하는 절차
- FP·FN 원인 분석
- 한계를 명시하는 태도

Step 4 예고 — 의료영상 질병 탐지

- 의료영상에서는 개인정보, 데이터 사용 절차, 임상 지표, 설명가능성이 중요
- 다음 실습에서는 공개 흉부 X선 데이터를 활용
- 데이터 사용 조건 및 라이선스를 사전에 확인